Step 4 — A Router That Sends Each Message to the Right Agent Automatically — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — What the router decides, and how
- Hint 2 — The plan, and where every piece moves

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — What the router decides, and how

The router answers one question per message: **Concierge or Triage?** A keyword list can't do this well — "the app freezes on login" has no complaint word, and "what's your refund policy?" has one but is just a question. So the router asks the model, and the model's answer must be one of two exact names.

Make the router a small structured-output call — the same `.parse(...)` you used for Triage, with a new schema:

```
class RouteDecision(BaseModel):
    agent: Literal["concierge", "triage"]
    reason: str
```

`decide(client, message) -> RouteDecision` in `agents/router.py`:

- Start with a safe default: `RouteDecision(agent="concierge", reason="router could not decide")`.
- Inside `try`, call `.parse(...)` with `prompts/router.txt` as the system prompt. If `reply.parsed is not None`, use it (it's `None` when the model refused).
- `except ValidationError` → log a warning and keep the default.
- Always end with one `logger.info(...)` line: agent, reason, message.

Why Concierge as the default: a wrong chat reply is harmless; a wrong ticket is not.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and where every piece moves

```
loop:
    message = input(...)
    decision = decide(client, message)
    if decision.agent == "triage": show triage_agent(client, message)
    else: concierge_agent(client, history, message)
```

For that, the chat code moves out of `main.py` into `agents/concierge.py`:

- `start_history()` returns `[system message from concierge.txt]`.
- `concierge_agent(client, history, message)` trims, then calls `stream_message()`.

`MAX_HISTORY_TOKENS` moves with it.

Put the flow in `services/desk_service.py` (plus an empty `services/__init__.py`):

```
handle_message(client, history, message):
    decision = decide(client, message)
    if decision.agent == "triage":
        ticket = triage_agent(client, message)
        None → "couldn't turn that into a ticket"; else print_ticket(ticket)
    else:
        print("Concierge: ", end="", flush=True)
        concierge_agent(client, history, message)
```

`print_ticket()` is already in this file from Step 3 — add `handle_message()` below it. `main.py` shrinks to: startup, `history = start_history()`, the input loop, and **Step 3's exact `except` blocks** around `handle_message(...)`. Remove `/extract`.

Last, add a `"routing"` list to `messages.json` — `{"message": ..., "expected": "concierge"}` items. Add a `test_routing()` that prints `OK` or `WRONG` for each one, and a unit case showing `RouteDecision(agent="billing", ...)` raises `ValidationError`.

Full solution: Show me the solution